Gender dynamics experiments

Eduardo García-Portugués, Carlos Vaquero

2026-08-03

```
Running experiment 3
```

Load libraries

```
# Load libraries
library(knitr)
library(dplyr)
library(recipes)
library(rsample)
library(yardstick)
library(glmnet)
library(ggplot2)
library(caret)
library(randomForest)
library(tibble)
sessionInfo()
```

```
R version 4.5.2 (2025-10-31)
Platform: aarch64-apple-darwin20
Running under: macOS Tahoe 26.6

Matrix products: default
BLAS:   /System/Library/Frameworks/Accelerate.framework/Versions/A/Frameworks/vecLib.framework/Versio
LAPACK: /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRlapack.dylib;  LAPACK ve

locale:
[1] C.UTF-8/C.UTF-8/C.UTF-8/C/C.UTF-8/C.UTF-8

time zone: Europe/Madrid
tzcode source: internal

attached base packages:
[1] stats     graphics  grDevices utils     datasets  methods   base
```

```
other attached packages:
 [1] tibble_3.3.1         randomForest_4.7-1.2 caret_7.0-1
 [4] lattice_0.22-9       ggplot2_4.0.3        glmnet_5.0
 [7] Matrix_1.7-6         yardstick_1.4.0      rsample_1.3.2
[10] recipes_1.3.2        dplyr_1.2.1          knitr_1.51

loaded via a namespace (and not attached):
 [1] gtable_0.3.6         shape_1.4.6.1        xfun_0.60
 [4] vctrs_0.7.3          tools_4.5.2          generics_0.1.4
 [7] stats4_4.5.2         parallel_4.5.2       ModelMetrics_1.2.2.2
[10] pkgconfig_2.0.3      data.table_1.18.4    RColorBrewer_1.1-3
[13] S7_0.2.2             lifecycle_1.0.5      stringr_1.6.0
[16] compiler_4.5.2       farver_2.1.2         codetools_0.2-20
[19] htmltools_0.5.9      class_7.3-23         yaml_2.3.12
[22] prodlim_2025.04.28   pillar_1.11.1        furrr_0.4.0
[25] tidyr_1.3.2          MASS_7.3-66          gower_1.0.2
[28] iterators_1.0.14     rpart_4.1.27         foreach_1.5.2
[31] nlme_3.1-170         parallelly_1.48.0    lava_1.8.2
[34] tidyselect_1.2.1     digest_0.6.39        stringi_1.8.7
[37] future_1.75.0        reshape2_1.4.5       purrr_1.2.2
[40] listenv_1.0.0        splines_4.5.2        fastmap_1.2.0
[43] grid_4.5.2           cli_3.6.6            magrittr_2.0.5
[46] survival_3.8-9       future.apply_1.20.2  withr_3.0.3
[49] scales_1.4.0         lubridate_1.9.5      timechange_0.4.0
[52] rmarkdown_2.31       globals_0.19.1       otel_0.2.0
[55] nnet_7.3-20          timeDate_4052.112    evaluate_1.0.5
[58] hardhat_1.4.3        rlang_1.3.0          Rcpp_1.1.2
[61] glue_1.8.1           pROC_1.19.0.1        ipred_0.9-15
[64] jsonlite_2.0.0       plyr_1.8.9           R6_2.6.1
```

Read dataset

```
# Load the single-source dataset, downloaded from Zenodo on first use. The file
# is not tracked by the repository; it is fetched once and reused thereafter.
# The literal NA values in Sex (arias with no singer record) become real NAs, and
# stringsAsFactors turns Gender/Sex and the four character-valued interval
# features into factors with alphabetical levels, so the second level is the
# masculine/male class in every experiment
data_file <- "sopranovoices.csv"
data_doi <- "10.5281/zenodo.21757127"
data_url <- paste0("https://zenodo.org/records/21757127/files/",
                   "sopranovoices.csv?download=1")
data_md5 <- "2b495d57628d67f962e8fe44fdfcb02d"
if (!file.exists(data_file)) {
```

```
  message("Downloading ", data_file, " from Zenodo (DOI ", data_doi, ")")
  # Raise the 60 s default timeout for this transfer only, and write to a
  # temporary file so an interrupted download never leaves a truncated
  # sopranovoices.csv that later runs would silently reuse
  old_opts <- options(timeout = max(600, getOption("timeout")))
  tmp_file <- paste0(data_file, ".part")
  download.file(url = data_url, destfile = tmp_file, mode = "wb")
  options(old_opts)
  if (!identical(unname(tools::md5sum(tmp_file)), data_md5)) {

    unlink(tmp_file)
    stop("Checksum mismatch for the file downloaded from ", data_url)

  }
  file.rename(tmp_file, data_file)

}

# Guarantee that every run uses the exact deposited file
stopifnot(identical(unname(tools::md5sum(data_file)), data_md5))
df_all <- read.csv(data_file, stringsAsFactors = TRUE)
stopifnot(nrow(df_all) == 1682, ncol(df_all) == 576)

# Metadata columns, never used as predictors
meta_cols <- c("AriaId", "ariaTitle", "AriaOpera", "Character", "Gender",
               "Composer", "Year", "Singer", "Sex")

# Experiment subsets and target y:
# 1 = all arias, y = character gender;
# 2 = arias whose character gender aligns with the singer's sex, y = gender;
# 3 = arias with a known singer, y = singer's sex
df <- switch(
  as.character(experiment),
  "1" = df_all |>
    mutate(y = Gender),
  "2" = df_all |>
    filter((Gender == "Feminine" & Sex == "Female") |
             (Gender == "Masculine" & Sex == "Male")) |>
    mutate(y = Gender),
  "3" = df_all |>
    filter(!is.na(Sex)) |>
    mutate(y = Sex),
  stop("Experiment ", experiment, " not supported (expected 1, 2 or 3)")
)

# Drop metadata and any factor levels emptied by the filters
df <- df |>
  select(-all_of(meta_cols)) |>
```

```
  droplevels()

# Number of observations per group
df |>
  group_by(y) |>
  summarise(n = n(), .groups = "drop")
```

```
# A tibble: 2 x 2
  y          n
  <fct>  <int>
1 Female   643
2 Male     793
```

```
# Split into train / test (70% / 30%), stratified by y so that both sets
# preserve the class balance of the full dataset.
set.seed(123)
split_obj <- initial_split(df, prop = 0.70, strata = y)
train <- training(split_obj)
test <- testing(split_obj)
```

Corpus vocal ranges

```
# Two-panel boxplots of the arias' vocal ranges: highest and lowest notes by
# premiering singer's sex (arias with a known singer) and by character gender
# (all arias). Independent of the current experiment, so it is rendered only
# in the experiment == 1 pass.
if (experiment == 1) {

  make_ranges <- function(data, group, panel) {
    bind_rows(
      tibble(group = group, note = "Highest notes",
             index = data$PartSop_HighestNoteIndex),
      tibble(group = group, note = "Lowest notes",
             index = data$PartSop_LowestNoteIndex)
    ) |>
      mutate(panel = panel)
  }
  singers <- df_all |> filter(!is.na(Sex))
  ranges_df <- bind_rows(
    make_ranges(singers, as.character(singers$Sex), "Singer"),
    make_ranges(df_all, as.character(df_all$Gender), "Character")
  ) |>
    mutate(panel = factor(panel, levels = c("Singer", "Character")))
```

```
  # C-note axis: note index 60 = C4, so octave number = index / 12 - 1
  c_breaks <- seq(48, 96, by = 12)
  plot_ranges <- ggplot(ranges_df, aes(x = group, y = index, fill = note)) +
    geom_boxplot(outlier.shape = 1, outlier.size = 0.8) +
    facet_wrap(~ panel, scales = "free_x", strip.position = "bottom") +
    scale_fill_manual(values = c("Highest notes" = "grey35",
                                 "Lowest notes" = "grey85")) +
    scale_y_continuous(breaks = c_breaks,
                       labels = paste0("C", c_breaks / 12 - 1),
                       limits = c(48, 96)) +
    labs(x = NULL, y = "Note", fill = NULL) +
    theme_bw() +
    theme(strip.placement = "outside",
          strip.background = element_blank(),
          axis.text.x = element_text(angle = 45, hjust = 1),
          legend.position = c(0.998, 0.995),
          legend.justification = c(1, 1),
          legend.key.size = grid::unit(0.85, "lines"),
          legend.text = element_text(size = 8),
          # Careful with randomForest::margin()
          legend.margin = ggplot2::margin(1, 3, 1, 1),
          legend.background = element_rect(fill = "white", colour = "grey60",
                                           linewidth = 0.3))
  plot_ranges

  # Save plot
  dir.create("plots", showWarnings = FALSE, recursive = TRUE)
  ggsave("plots/singers_vs_characters_ranges.png", plot = plot_ranges,
         width = 7.5, height = 2.9, dpi = 300, bg = "white")

}
```

Dataset summary tables

```
# Sources for the corpus-description tables of the paper: character gender vs.
# premiering singer's sex (Table 1) and the per-character breakdown (Table 2).
# Independent of the current experiment, so written only in the
# experiment == 1 pass.
if (experiment == 1) {

  dir.create("results/dataset_summary", showWarnings = FALSE, recursive = TRUE)

  # Table 1: distribution of arias by character gender and singer sex
  singer_col <- factor(
    ifelse(is.na(df_all$Sex), "Unknown singers",
```

```
          paste(df_all$Sex, "singers")),
    levels = c("Male singers", "Female singers", "Unknown singers")
  )
  table1 <- as.data.frame.matrix(table(df_all$Gender, singer_col)) |>
    rownames_to_column("Gender")
  write.csv(table1, "results/dataset_summary/table1_distribution.csv",
            row.names = FALSE)
  table1 |>
    kable()

  # Table 2: per-character singer-sex breakdown
  table2 <- df_all |>
    group_by(Character) |>
    summarise(
      Gender = first(Gender),
      female_ratio = ifelse(sum(!is.na(Sex)) > 0,
                            sum(Sex == "Female", na.rm = TRUE) /
                              sum(!is.na(Sex)), NA),
      male_ratio = ifelse(sum(!is.na(Sex)) > 0,
                          sum(Sex == "Male", na.rm = TRUE) /
                            sum(!is.na(Sex)), NA),
      known = sum(!is.na(Sex)),
      total = n(),
      .groups = "drop"
    ) |>
    arrange(desc(known), desc(total))
  write.csv(table2, "results/dataset_summary/table2_characters.csv",
            row.names = FALSE)
  table2 |>
    kable(digits = 2)

}
```

```
# Feature selection is name-based (deterministic given the column names)
selected_features <- select_sop_features(train)
stopifnot(length(selected_features) == 86)
train <- train |>
  select(all_of(selected_features), y)
test <- test |>
  select(all_of(selected_features), y)
dim(train)
```

```
[1] 1005   87
```

```
dim(test)
```

```
[1] 431  87
```

```r
# Full dataset for the final-model recipe: all observations in their original
# order (train and test are a partition of df, but their concatenation would
# reorder the rows and hence change the CV folds of the final ridge fit)
combined <- df |>
  select(all_of(selected_features), y)
```

Preprocessing recipes

```r
# Preprocessing recipe
preprocess_recipe <- function(data) {
  recipe(y ~ ., data = data) |>
    # Filter and impute all predictors BEFORE dummying.
    step_zv(all_predictors()) |>
    step_nzv(all_predictors()) |>
    step_filter_missing(all_predictors(), threshold = 0.05) |>
    step_impute_knn(all_predictors(), neighbors = 5) |>
    # Collapse rare factor levels (BEFORE dummy creation)
    step_other(all_nominal_predictors(), threshold = 0.10) |>
    # Decorrelate numeric predictors only
    step_corr(all_numeric_predictors(), threshold = 0.80) |>
    step_lincomb(all_numeric_predictors(), max_steps = 20) |>
    # Create dummy variables. With one_hot = TRUE every
    # level surviving step_other() gets an indicator; dropping "_other" below
    # then makes the pooled rare levels the reference
    step_dummy(all_nominal_predictors(), one_hot = TRUE) |>
    # Drop the dummies for the "other" level, making it the reference
    step_rm(ends_with("_other")) |>
    # Post-dummy cleanup
    step_zv(all_predictors()) |>
    step_nzv(all_predictors()) |>
    step_lincomb(all_predictors(), max_steps = 20) |>
    # Standardize LAST so the dummy indicators are standardized too and every
    # column enters glmnet on unit-sd scale
    step_normalize(all_numeric_predictors())
}
```

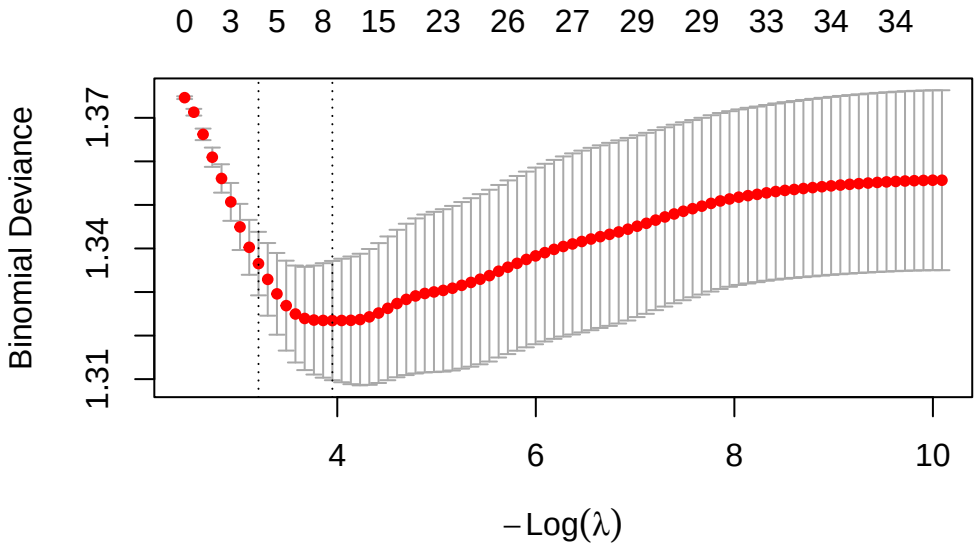
```r
# Train recipe using train data
rec_train <- preprocess_recipe(data = train)
prep_rec_train <- prep(rec_train)

# Bake train data
train_bake <- bake(prep_rec_train, new_data = NULL)
test_train_bake <- bake(prep_rec_train, new_data = test)

# Train recipe using combined data (train + test)
```

```r
rec_all <- preprocess_recipe(data = combined)
prep_all <- prep(rec_all)

# Fully baked dataset for the final model fit
all_bake <- bake(prep_all, new_data = NULL)

# Export datasets for future reference and use
dir.create("preprocessed_datasets", showWarnings = FALSE, recursive = TRUE)
write.csv(train_bake, paste0("preprocessed_datasets/train_bake_exp",
                             experiment, ".csv"),
          row.names = FALSE)
write.csv(test_train_bake, paste0("preprocessed_datasets/test_train_bake_exp",
                                  experiment, ".csv"),
          row.names = FALSE)
write.csv(all_bake, paste0("preprocessed_datasets/all_bake_exp", experiment,
                           ".csv"),
          row.names = FALSE)
```

Lasso logistic

Fitting

```r
# Fit by 10-fold cross-validation, with folds stratified by y and shared
# between the lasso and ridge fits so that both models see the same partitions
set.seed(42)
train_bake_no_y <- train_bake |>
  select(-y) |>
  as.matrix()
foldid_train <- caret::createFolds(train_bake$y, k = n_folds, list = FALSE)
cv1_train <- cv.glmnet(x = train_bake_no_y, y = train_bake$y,
                       family = "binomial", alpha = 1, foldid = foldid_train)
plot(cv1_train)
```



Predictions

```
# Predictions
test_train_bake_no_y <- test_train_bake |>
  select(-y) |>
  as.matrix()
y_test_hat_LL <- predict(cv1_train, newx = test_train_bake_no_y,
                         s = "lambda.1se", type = "class")
p_test_hat_LL <- predict(cv1_train, newx = test_train_bake_no_y,
                         s = "lambda.1se", type = "response")
```

Metrics

```
# Metrics
acc_LL <- mean(test$y == y_test_hat_LL)
auc_LL <- auc_binary(test$y, p_test_hat_LL)
f1_LL <- f1(test$y, y_test_hat_LL)
data.frame("Accuracy" = acc_LL, "AUC" = auc_LL, "Macro_F1" = f1_LL) |>
  kable(digits = 4)
```

Accuracy	AUC	Macro_F1
0.5731	0.6098	0.4888

Relevant coefficients

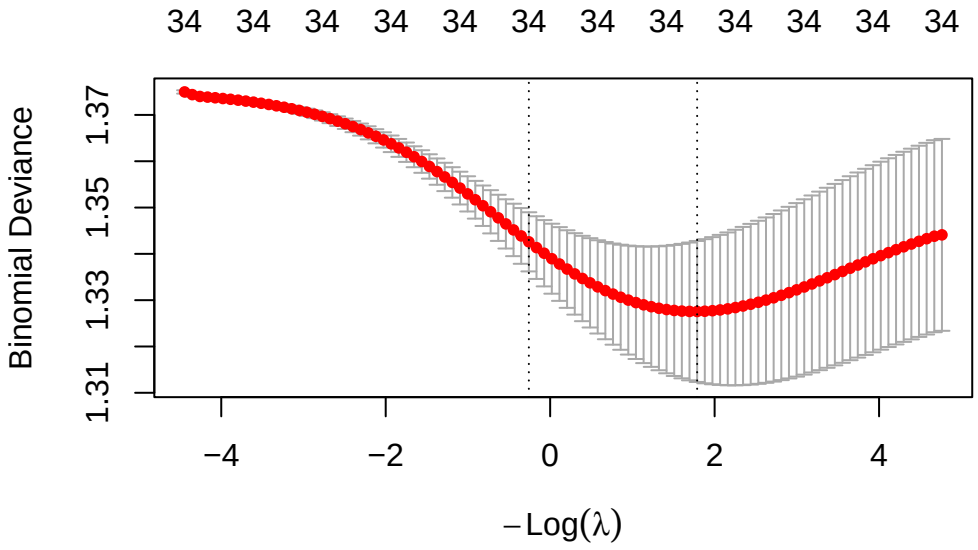
```
# Extract nonzero coefficients for lambda.1se and order them by absolute value
coefs1_train <- predict(cv1_train, s = "lambda.1se", type = "coefficients") |>
  as.matrix() |>
  as.data.frame() |>
  rownames_to_column(var = "feature") |>
  rename("coefficient" = "lambda.1se")
coefs1_train |>
  filter(coefficient != 0) |>
  arrange(desc(abs(coefficient))) |>
  kable(digits = 4)
```

feature	coefficient
(Intercept)	0.2137
PartSop_LargestSemitonesAsc	0.1722
PartSop_DescendingIntervallicStd	0.1358
PartSop_HighestNoteIndex	-0.0883

Ridge logistic model

Fitting

```
# Fit by 10-fold cross-validation, reusing the stratified folds of the lasso
cv0_train <- cv.glmnet(x = train_bake_no_y, y = train_bake$y,
                       family = "binomial", alpha = 0, foldid = foldid_train)
plot(cv0_train)
```



Predictions

```
# Predictions
y_test_hat_RL <- predict(cv0_train, newx = test_train_bake_no_y,
                         s = "lambda.1se", type = "class")
p_test_hat_RL <- predict(cv0_train, newx = test_train_bake_no_y,
                         s = "lambda.1se", type = "response")
```

Metrics

```
# Metrics
acc_RL <- mean(test$y == y_test_hat_RL)
auc_RL <- auc_binary(test$y, p_test_hat_RL)
f1_RL <- f1(test$y, y_test_hat_RL)
data.frame("Accuracy" = acc_RL, "AUC" = auc_RL, "Macro_F1" = f1_RL) |>
  kable(digits = 4)
```

Accuracy	AUC	Macro_F1
0.5847	0.602	0.4764

Relevant coefficients

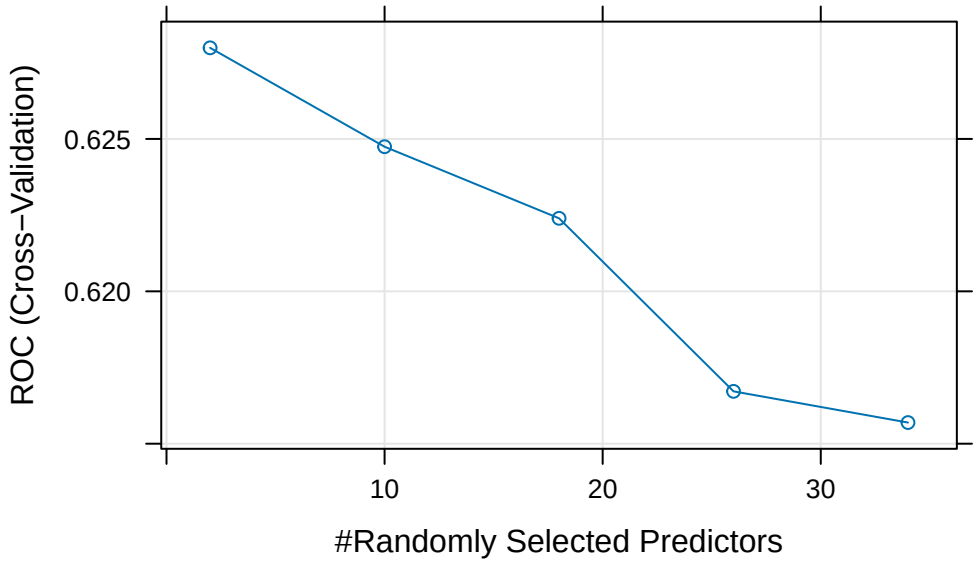
```r
# Extract coefficients and sort them by absolute value
coefs0_train <- predict(cv0_train, s = "lambda.1se", type = "coefficients") |>
  as.matrix() |>
  as.data.frame() |>
  rownames_to_column(var = "feature") |>
  rename("coefficient" = "lambda.1se")
coefs0_train |>
  filter(coefficient != 0) |>
  arrange(desc(abs(coefficient))) |>
  head(30) |>
  kable(digits = 4)
```

feature	coefficient
(Intercept)	0.2125
PartSop_LargestSemitonesAsc	0.0402
PartSop_HighestNoteIndex	-0.0397
PartSop_DescendingIntervallicStd	0.0362
PartSop_LargestSemitonesDesc	-0.0303
PartSop_AbsoluteIntervallicTrimRatio	0.0300
PartSop_Ambitus	0.0285
PartSop_IntervalsBeyondOctaveAsc_Per	0.0273
PartSop_IntervalsMinorAll_Per	-0.0252
PartSop_TrimmedAbsoluteIntervallicMean	0.0241
PartSop_IntervallicKurtosis	0.0219
PartSop_IntervalsPerfectDesc_Per	0.0212
PartSop_LargestIntervalAll_P8	-0.0192
PartSop_DottedRhythm	-0.0172
PartSop_IntervalsBeyondOctaveDesc_Per	0.0165
PartSop_IntervalsPerfectAsc_Per	0.0161
PartSop_IntervalsAugmentedAsc_Per	0.0148
PartSop_LeapsAsc_Per	0.0131
PartSop_IntervalsPerfectAll_Per	0.0130
PartSop_StepwiseMotionAll_Per	-0.0121
PartSop_AverageDuration	0.0120
PartSop_LeapsDesc_Per	0.0119
PartSop_IntervallicTrimRatio	-0.0116
PartSop_IntervalsDiminishedDesc_Per	-0.0114
PartSop_LargestIntervalAll_M9	0.0088
PartSop_IntervalsAugmentedDesc_Per	0.0083
PartSop_VoicePresence	0.0072
PartSop_SyllabicRatio	-0.0051
PartSop_RhythmInt	0.0048
PartSop_IntervalsDiminishedAsc_Per	0.0044

Random forests

Fitting

```
# Set seeds for reproducibility in caret with RF. caret expects a list of
# length n_folds + 1: one integer vector per fold (as long as the tuning grid)
# plus a final seed for the fit on the full training set.
set.seed(42)
rf_seeds <- c(
  lapply(seq_len(n_folds), function(i) rep(i, n_tune)),
  list(999L)
)

# Define the cross-validation scheme
cv_ctrl <- trainControl(method = "cv", number = n_folds, classProbs = TRUE,
                        summaryFunction = twoClassSummary,
                        savePredictions = "final", seeds = rf_seeds)

# Train Random Forest with tuning of mtry optimizing AUC
rf_train <- train(x = train_bake_no_y, y = train_bake$y, method = "rf",
                  metric = "ROC", trControl = cv_ctrl, tuneLength = n_tune)
plot(rf_train)
```



Predictions

```r
# Predictions
y_test_hat_RF <- predict(rf_train, newdata = test_train_bake_no_y)
positive_class <- levels(train_bake$y)[2] # "Class 1" is the second factor level
p_test_hat_RF <- predict(rf_train, newdata = test_train_bake_no_y,
                         type = "prob")[, positive_class]
```

Metrics

```r
# Metrics
acc_RF <- mean(test$y == y_test_hat_RF)
auc_RF <- auc_binary(test$y, p_test_hat_RF)
f1_RF <- f1(test$y, y_test_hat_RF)
data.frame("Accuracy" = acc_RF, "AUC" = auc_RF, "Macro_F1" = f1_RF) |>
  kable(digits = 4)
```

Accuracy	AUC	Macro_F1
0.5847	0.6069	0.5611

Variables importance

```r
# Variables importance
rf_importance <- varImp(rf_train)$importance
rf_importance |>
  rownames_to_column("feature") |>
  arrange(desc(Overall)) |>
  head(30) |>
  kable(digits = 4)
```

feature	Overall
PartSop_DescendingIntervallicStd	100.0000
PartSop_IntervalsMinorAll_Per	95.4261
PartSop_AbsoluteIntervallicTrimRatio	91.4357
PartSop_TrimmedAbsoluteIntervallicMean	90.0908
PartSop_IntervallicKurtosis	89.1263
PartSop_IntervalsPerfectAsc_Per	88.8473
PartSop_RhythmInt	87.5996
PartSop_IntervalsPerfectAll_Per	85.7160
PartSop_IntervalsPerfectDesc_Per	85.5289
PartSop_VoicePresence	85.2173
PartSop_LeapsAsc_Per	85.0394

feature	Overall
PartSop_IntervalsMajorAll_Per	83.8266
PartSop_StepwiseMotionAll_Per	82.5298
PartSop_IntervallicTrimDiff	82.3125
PartSop_SyllabicRatio	82.1177
PartSop_LeapsDesc_Per	82.0150
PartSop_IntervalsMajorAsc_Per	81.1559
PartSop_AverageDuration	80.7021
PartSop_IntervalsWithinOctaveDesc_Per	77.4982
PartSop_IntervallicTrimRatio	73.9312
PartSop_IntervallicMean	67.7330
PartSop_HighestNoteIndex	65.2745
PartSop_DottedRhythm	64.7406
PartSop_Ambitus	62.4062
PartSop_IntervalsAugmentedAsc_Per	60.4447
PartSop_IntervalsBeyondOctaveAsc_Per	56.5423
PartSop_IntervalsDiminishedDesc_Per	54.7679
PartSop_IntervalsDiminishedAsc_Per	52.7296
PartSop_LargestSemitonesAsc	52.3208
PartSop_LargestSemitonesDesc	44.2141

Metrics for all models on test data

```
# Save predictions for all models
dir.create("results/y_hats", showWarnings = FALSE, recursive = TRUE)
datasets_to_save <- list(
  RL = y_test_hat_RL,
  LL = y_test_hat_LL,
  RF = y_test_hat_RF,
  y_true = test$y
)
for (name in names(datasets_to_save)) {

  # Coerce to a one-column data frame with a stable column name. Note that
  # is.vector() is FALSE for both the glmnet class-prediction matrices and the
  # y_true factor, so the labels have to be extracted with as.vector().
  data_to_write <- data.frame(
    prediction = as.vector(datasets_to_save[[name]])
  )

  # Construct filename
  file_name <- paste0("results/y_hats/", name, "_", experiment, ".csv")

  # Write CSV
  write.csv(data_to_write, file_name, row.names = FALSE)
```

```
}
```

```
# Combine all metrics into a single table
metrics_table <- data.frame(
  Accuracy = c(acc_RL, acc_LL, acc_RF),
  AUC = c(auc_RL, auc_LL, auc_RF),
  Macro_F1 = c(f1_RL, f1_LL, f1_RF),
  Model = c("Ridge Logistic (RL)", "Lasso Logistic (LL)",
            "Random Forest (RF)")
)
metrics_table |>
  kable(digits = 4)
```

Accuracy	AUC	Macro_F1	Model
0.5847	0.6020	0.4764	Ridge Logistic (RL)
0.5731	0.6098	0.4888	Lasso Logistic (LL)
0.5847	0.6069	0.5611	Random Forest (RF)

Bootstrap tests for model comparison

```
# Run bootstrapping of accuracy with predictions
boot_RL <- boot_acc(test$y, y_test_hat_RL, n_boot = B, seed = 123)
boot_LL <- boot_acc(test$y, y_test_hat_LL, n_boot = B, seed = 123)
boot_RF <- boot_acc(test$y, y_test_hat_RF, n_boot = B, seed = 123)
boot <- list(RL = boot_RL, LL = boot_LL, RF = boot_RF)

# Bootstrap accuracy differences between RF and other models
diff_boot_RF_RL <- boot$RF - boot$RL
diff_boot_RF_LL <- boot$RF - boot$LL

# One-sided p-values (H1: RF better than other model)
p_RF_vs_RL <- (1 + sum(diff_boot_RF_RL <= 0)) / (B + 1)
p_RF_vs_LL <- (1 + sum(diff_boot_RF_LL <= 0)) / (B + 1)

# Two-sided confidence intervals for differences
ci_diff_RL <- quantile(diff_boot_RF_RL, probs = ci_probs)
ci_diff_LL <- quantile(diff_boot_RF_LL, probs = ci_probs)

# Add bootstrap tests to metrics table
metrics_table <- metrics_table |>
  mutate(
    ci_diff_lo = c(ci_diff_RL[1], ci_diff_LL[1], NA),
    ci_diff_up = c(ci_diff_RL[2], ci_diff_LL[2], NA),
```

```
    pval_better = c(p_RF_vs_RL, p_RF_vs_LL, NA)
  )
metrics_table |>
  kable(digits = 4)
```

Accuracy	AUC	Macro_F1	Model	ci_diff_lo	ci_diff_up	pval_better
0.5847	0.6020	0.4764	Ridge Logistic (RL)	-0.0464	0.0465	0.5175
0.5731	0.6098	0.4888	Lasso Logistic (LL)	-0.0348	0.0580	0.3289
0.5847	0.6069	0.5611	Random Forest (RF)	NA	NA	NA

Comparison with majority class baseline

```
# Majority Class (MC) prediction baseline - always predict most common class
class_probs <- table(train$y) / length(train$y)
majority_class_train <- names(sort(class_probs, decreasing = TRUE))[1]
y_test_hat_MC <- factor(rep(majority_class_train, length(test$y)),
                        levels = levels(test$y))

# Metrics for majority class baseline
acc_MC <- mean(test$y == y_test_hat_MC)
p_test_hat_MC <- rep(class_probs[majority_class_train], length(test$y))
auc_MC <- auc_binary(test$y, p_test_hat_MC)
f1_MC <- f1(test$y, y_test_hat_MC)

# Bootstrap accuracy for majority class baseline
boot_MC <- boot_acc(test$y, y_test_hat_MC, n_boot = B, seed = 123)

# Bootstrap accuracy differences between majority class baseline and models
diff_boot_RL_MC <- boot_RL - boot_MC
diff_boot_LL_MC <- boot_LL - boot_MC
diff_boot_RF_MC <- boot_RF - boot_MC

# One-sided p-values (H1: model better than majority class baseline)
p_RL_vs_MC <- (1 + sum(diff_boot_RL_MC <= 0)) / (B + 1)
p_LL_vs_MC <- (1 + sum(diff_boot_LL_MC <= 0)) / (B + 1)
p_RF_vs_MC <- (1 + sum(diff_boot_RF_MC <= 0)) / (B + 1)

# Two-sided confidence intervals for differences
ci_diff_RL_MC <- quantile(diff_boot_RL_MC, probs = ci_probs)
ci_diff_LL_MC <- quantile(diff_boot_LL_MC, probs = ci_probs)
ci_diff_RF_MC <- quantile(diff_boot_RF_MC, probs = ci_probs)

# Combine all metrics into a single table
metrics_table_MC <- data.frame(
```

```
  Accuracy = c(acc_RL, acc_LL, acc_RF, acc_MC),
  AUC = c(auc_RL, auc_LL, auc_RF, auc_MC),
  Macro_F1 = c(f1_RL, f1_LL, f1_RF, f1_MC),
  ci_diff_lo = c(ci_diff_RL_MC[1], ci_diff_LL_MC[1], ci_diff_RF_MC[1], NA),
  ci_diff_up = c(ci_diff_RL_MC[2], ci_diff_LL_MC[2], ci_diff_RF_MC[2], NA),
  pval_better = c(p_RL_vs_MC, p_LL_vs_MC, p_RF_vs_MC, NA),
  Model = c("Ridge Logistic (RL)", "Lasso Logistic (LL)",
            "Random Forest (RF)", "Majority Class (MC)")
)
metrics_table_MC |>
  kable(digits = 4)
```

Accuracy	AUC	Macro_F1	ci_diff_lo	ci_diff_up	pval_better	Model
0.5847	0.6020	0.4764	0.0023	0.0626	0.0172	Ridge Logistic (RL)
0.5731	0.6098	0.4888	-0.0162	0.0580	0.1410	Lasso Logistic (LL)
0.5847	0.6069	0.5611	-0.0209	0.0858	0.1231	Random Forest (RF)
0.5522	0.5000	0.3558	NA	NA	NA	Majority Class (MC)

```
# In a combined table
metrics_table_combined <- metrics_table_MC |>
  mutate(ci_diff_lo_MC = ci_diff_lo, ci_diff_up_MC = ci_diff_up,
         pval_better_MC = pval_better) |>
  mutate(ci_diff_lo_RF = c(metrics_table$ci_diff_lo, NA),
         ci_diff_up_RF = c(metrics_table$ci_diff_up, NA),
         pval_better_RF = c(metrics_table$pval_better, NA)) |>
  select(Model, Accuracy, AUC, Macro_F1,
         ci_diff_lo_RF, ci_diff_up_RF, pval_better_RF,
         ci_diff_lo_MC, ci_diff_up_MC, pval_better_MC)
metrics_table_combined |>
  kable(digits = 4)
```

Model	Accuracy	AUC	Macro_F1	ci_diff_lo_RF	ci_diff_up_RF	pval_better_RF	ci_diff_lo_MC	ci_diff_up_MC	pval_better_MC
Ridge Logistic (RL)	0.5847	0.6020	0.4764	-0.0464	0.0465	0.5175	0.0023	0.0626	0.0172
Lasso Logistic (LL)	0.5731	0.6098	0.4888	-0.0348	0.0580	0.3289	-0.0162	0.0580	0.1410
Random Forest (RF)	0.5847	0.6069	0.5611	NA	NA	NA	-0.0209	0.0858	0.1231
Majority Class (MC)	0.5522	0.5000	0.3558	NA	NA	NA	NA	NA	NA

```r
# Save table of all metrics + bootstrap tests
dir.create("results/experiments", showWarnings = FALSE, recursive = TRUE)
write.csv(metrics_table_combined, paste0("results/experiments/all_results_",
                                         "Experiment_", experiment, ".csv"),
          row.names = FALSE)
```

```r
# Print LaTeX table
latex_table <- metrics_table_combined |>
  mutate(ci_RF = paste0("$(", sprintf("%.4f", ci_diff_lo_RF), ", ",
                        sprintf("%.4f", ci_diff_up_RF), ")$"),
         ci_MC = paste0("$(", sprintf("%.4f", ci_diff_lo_MC), ", ",
                        sprintf("%.4f", ci_diff_up_MC), ")$"),
         p_RF = sprintf("%.4f", pval_better_RF),
         p_MC = sprintf("%.4f", pval_better_MC)) |>
  select(Model, Accuracy, AUC, Macro_F1, ci_RF, p_RF, ci_MC, p_MC)
latex_table |>
  kable(digits = 4)
```

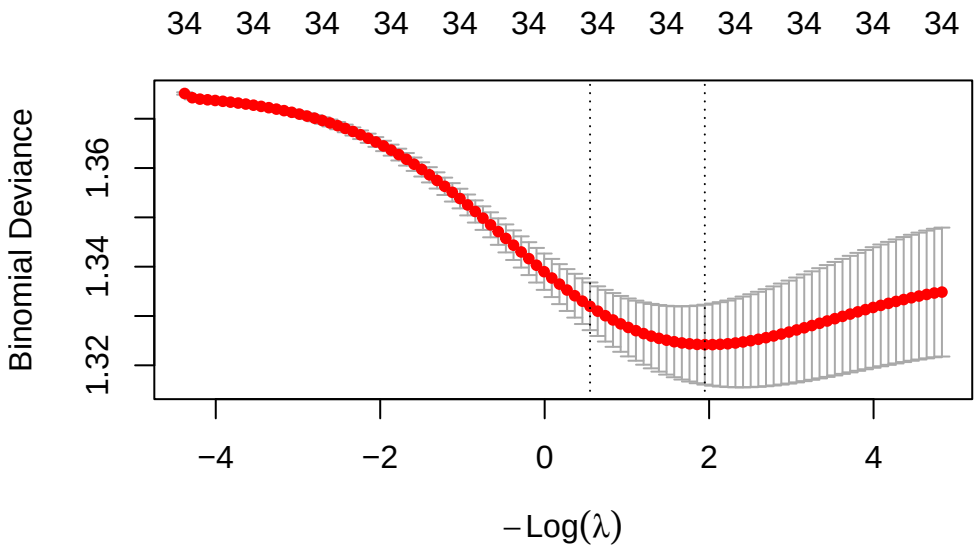
Model	Accuracy	AUC	Macro_F1	ci_RF	p_RF	ci_MC	p_MC
Ridge Logistic (RL)	0.5847	0.6020	0.4764	$(-0.0464, 0.0465)$	0.5175	$(0.0023, 0.0626)$	0.0172
Lasso Logistic (LL)	0.5731	0.6098	0.4888	$(-0.0348, 0.0580)$	0.3289	$(-0.0162, 0.0580)$	0.1410
Random Forest (RF)	0.5847	0.6069	0.5611	(NA, NA)	NA	$(-0.0209, 0.0858)$	0.1231
Majority Class (MC)	0.5522	0.5000	0.3558	(NA, NA)	NA	(NA, NA)	NA

```r
latex_table |>
  kable(digits = 4, format = "latex", booktabs = TRUE, escape = FALSE) |>
  print()
```

```
\begin{tabular}{lrrrllll}
\toprule
Model & Accuracy & AUC & Macro_F1 & ci_RF & p_RF & ci_MC & p_MC\\
\midrule
Ridge Logistic (RL) & 0.5847 & 0.6020 & 0.4764 & $(-0.0464, 0.0465)$ & 0.5175 & $(0.0023, 0.0626)$ & 0.0172\\
Lasso Logistic (LL) & 0.5731 & 0.6098 & 0.4888 & $(-0.0348, 0.0580)$ & 0.3289 & $(-0.0162, 0.0580)$ & 0.1410\\
Random Forest (RF) & 0.5847 & 0.6069 & 0.5611 & $(NA, NA)$ & NA & $(-0.0209, 0.0858)$ & 0.1231\\
Majority Class (MC) & 0.5522 & 0.5000 & 0.3558 & $(NA, NA)$ & NA & $(NA, NA)$ & NA\\
\bottomrule
\end{tabular}
```

Ridge logistic for inference and final model on all data (train + test)

```r
# Fit ridge
set.seed(42)
all_bake_no_y <- all_bake |>
  select(-y) |>
  as.matrix()
foldid_all <- caret::createFolds(all_bake$y, k = n_folds, list = FALSE)
cv0_all <- cv.glmnet(x = all_bake_no_y, y = all_bake$y, family = "binomial",
                     alpha = 0, foldid = foldid_all)
plot(cv0_all)
```




```
cv0_all
```

```
Call:  cv.glmnet(x = all_bake_no_y, y = all_bake$y, foldid = foldid_all,      family = "binomial", al

Measure: Binomial Deviance

    Lambda Index Measure       SE Nonzero
min 0.1427    69   1.324 0.008099      34
1se 0.5759    54   1.332 0.004833      34
```

```
# Bonferroni-corrected confidence intervals for all coefficients; the family
# is the p slopes of this case study (the intercept keeps a marginal CI)
p_slopes <- ncol(all_bake_no_y)
ci_df_all <- logistic_ridge_ci(X = all_bake_no_y,
                               beta_hat = coef(cv0_all, s = cv0_all$lambda.1se),
                               lambda = nrow(all_bake) * cv0_all$lambda.1se / 2,
                               level = level, m = p_slopes)

# Effect sizes as percent changes in odds, exported to CSV for the paper
ci_df_all <- ci_df_all |>
  mutate(
    pct_change = (exp(estimate) - 1) * 100,
    pct_low = (exp(ci_low) - 1) * 100,
    pct_up = (exp(ci_up) - 1) * 100
  )
dir.create("results/final_model", showWarnings = FALSE, recursive = TRUE)
write.csv(ci_df_all, paste0("results/final_model/ci_df_Experiment_",
                            experiment, ".csv"),
          row.names = FALSE)
```

```r
# Significant slopes with and without the Bonferroni correction, for reference
slopes_df <- ci_df_all |>
  filter(term != "(Intercept)")
zcrit_marg <- qnorm(1 - (1 - level) / 2)
data.frame(
  Rule = c("Unadjusted (marginal CI excludes 0)",
           "Bonferroni (simultaneous CI excludes 0)"),
  N_selected = c(sum(abs(slopes_df$z_value) > zcrit_marg),
                 sum(sign(slopes_df$ci_low * slopes_df$ci_up) > 0)),
  N_total = nrow(slopes_df)
) |>
  kable()
```

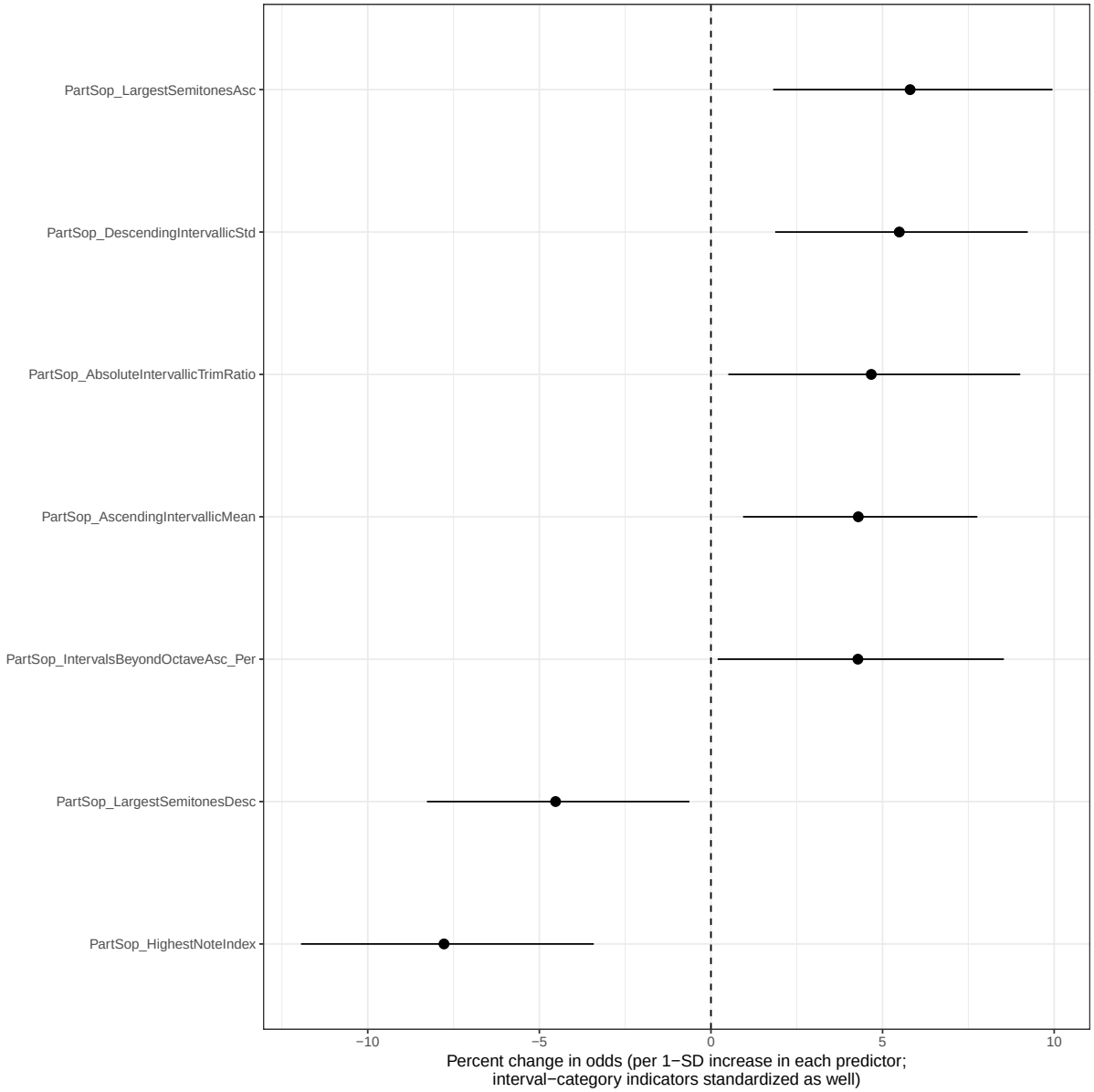
Rule	N_selected	N_total
Unadjusted (marginal CI excludes 0)	12	34
Bonferroni (simultaneous CI excludes 0)	7	34

```r
# Filter significant slopes (corrected CI excluding zero), order by effect size
ci_df <- slopes_df |>
  filter(sign(ci_low * ci_up) > 0) |>
  mutate(
    term = factor(term, levels = term[order(pct_change, decreasing = FALSE)]
    )
  )
ci_df |>
  select(term, pct_change, pct_low, pct_up, p_value, p_value_adj) |>
  arrange(desc(pct_change)) |>
  kable(digits = 4)
```

term	pct_change	pct_low	pct_up	p_value	p_value_adj
PartSop_LargestSemitonesAsc	5.8026	1.8123	9.9492	0e+00	0.0001
PartSop_DescendingIntervallicStd	5.4870	1.8721	9.2303	0e+00	0.0000
PartSop_AbsoluteIntervallicTrimRatio	4.6710	0.5046	9.0101	4e-04	0.0119
PartSop_AscendingIntervallicMean	4.2923	0.9349	7.7615	0e+00	0.0015
PartSop_IntervalsBeyondOctaveAsc_Per	4.2809	0.1939	8.5346	9e-04	0.0291
PartSop_LargestSemitonesDesc	-4.5278	-8.2753	-0.6272	2e-04	0.0079
PartSop_HighestNoteIndex	-7.7784	-11.9451	-3.4145	0e+00	0.0000

```r
# Plot of coefficients in odds
plot_RL <- ggplot(ci_df, aes(x = term, y = pct_change,
                             ymin = pct_low, ymax = pct_up)) +
  geom_pointrange() +
```

```
  geom_hline(yintercept = 0, linetype = 2) +
  coord_flip() +
  labs(
    x = NULL,
    y = paste0("Percent change in odds (per 1-SD increase in each predictor;",
               "\ninterval-category indicators standardized as well)")
  ) +
  theme_bw()
plot_RL
```



```r
# Save plot
dir.create("plots", showWarnings = FALSE, recursive = TRUE)
plot_name <- paste0("plots/plot_rl_", experiment, ".png")
ggsave(filename = plot_name, plot = plot_RL, width = 7.5, height = 6,
       dpi = 300)

# Metrics on all data available (in-sample), just for reference
y_all_hat_RL <- predict(cv0_all, newx = all_bake_no_y, s = "lambda.1se",
                        type = "class")
p_all_hat_RL <- predict(cv0_all, newx = all_bake_no_y, s = "lambda.1se",
                        type = "response")
acc_all_RL <- mean(all_bake$y == y_all_hat_RL)
auc_all_RL <- auc_binary(all_bake$y, p_all_hat_RL)
f1_all_RL <- f1(all_bake$y, y_all_hat_RL)
glmnet_summary <- data.frame("Accuracy" = acc_all_RL, "AUC" = auc_all_RL,
                             "Macro_F1" = f1_all_RL,
                             "Model" = "Ridge Logistic (RL)",
                             "Evaluation" = "In-sample (train + test)")
glmnet_summary |>
  kable(digits = 4)
```

Accuracy	AUC	Macro_F1	Model	Evaluation
0.6079	0.6495	0.5625	Ridge Logistic (RL)	In-sample (train + test)

```r
# Save results
dir.create("results/final_model", showWarnings = FALSE, recursive = TRUE)
write.csv(glmnet_summary, paste0("results/final_model/RL_final_model_results",
                                 "_Experiment_", experiment, ".csv"),
          row.names = FALSE)
```